Lab Assignment 1

Notes: **Shape of the dataset is (9450,13)**

[Note if you are not getting this shape that means your data has not been uploaded correctly to the colab]

The target feature in the dataset is the **'Price'** column. Random state should be taken as **42** wherever applicable. Ignore all the warnings while executing the codes.

MetaData

1. Airline:The name of the airline
2. Source: The source from which the service begins
3. Destination: The destination where the service ends
4. Route: Route the flight took.
5. Dep_Time: The time when the journey starts from the source.
6. Arrival_Time: Time of arrival at the destination.
7. Duration: Total duration of the flight.
8. Total_Stops: Total stops between the source and destination.
9. Additional_Info: Additional information about the flight
10. Price: The price of the ticket ▶▶▶ Target
11. Month: Month of journey.
12. WeekDay: Day at which journey started.
13. Day: Date of the start of journey.

Click here to view the dataset: Dataset

Note: The below mentioned Step 1 and Step 2 are standard import of google drive and loading of a dataset from google drive. You can also load the dataset locally if you wish to.

Step 1: Loading the dataset and importing the necessary libraries

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Step 2 : Verify the shape of the dataset

```
import pandas as pd
import numpy as np
```

```
dataset_path = 'Preprocessing1.csv'
df = pd.read_csv(dataset_path)
```

```
df.sample(5)
```

	Airline	Source	Destination	Route	Dep_Time	Arrival_Time	Duration
3469	IndiGo	Kolkata	Banglore	CCU → BLR	11:30	14:05	2h 35m
154	SpiceJet	Kolkata	New Delhi	CCU → BLR	11:40	00:40 04 Jun	22h 45m
				BLR			

```
df.shape
```

```
(9450, 13)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9450 entries, 0 to 9449
Data columns (total 13 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   Airline          9450 non-null   object 
 1   Source           9450 non-null   object 
 2   Destination      9450 non-null   object 
 3   Route            9449 non-null   object 
 4   Dep_Time         9450 non-null   object 
 5   Arrival_Time     9450 non-null   object 
 6   Duration         9450 non-null   object 
 7   Total_Stops      9250 non-null   object 
 8   Additional_Info  9450 non-null   object 
 9   Price            9450 non-null   int64  
 10  Month            9450 non-null   int64  
 11  WeekDay          9450 non-null   object 
 12  Day              9214 non-null   float64
dtypes: float64(1), int64(2), object(10)
memory usage: 959.9+ KB
```

```
df.describe()
```

	Price	Month	Day
count	9450.000000	9450.000000	9214.000000
mean	9027.895556	4.718730	13.517582
std	4466.677471	1.162725	8.459792
min	1759.000000	3.000000	1.000000
25%	5198.000000	3.000000	6.000000
50%	8366.000000	5.000000	12.000000
75%	12373.000000	6.000000	21.000000
max	57209.000000	6.000000	27.000000

Q1. What is the average of the flight ticket price? Write your answer correct to two decimal places.

```
avg_price = df.describe()['Price']['mean']
np.round(avg_price, 2)
```

```
np.float64(9027.9)
```

Answer: Average of the flight ticket price = **9027.9**

Q2. During which month did the highest number of flights occur? Months are represented by numerical codes, with January corresponding to 1, February to 2, and so forth.

```
df.describe().get('Month')['max']
```

```
np.float64(6.0)
```

Answer: Month has highest number of flights: **June**

Q3. Is the average price of flight tickets higher on weekends (Saturday and Sunday) or on weekdays (Remaining 5 days)?

```
df.describe()
```

	Price	Month	Day
count	9450.000000	9450.000000	9214.000000
mean	9027.895556	4.718730	13.517582
std	4466.677471	1.162725	8.459792
min	1759.000000	3.000000	1.000000
25%	5198.000000	3.000000	6.000000
50%	8366.000000	5.000000	12.000000
75%	12373.000000	6.000000	21.000000
max	57209.000000	6.000000	27.000000

```
# Getting all flights on weekends
weekend_flights = df[df['WeekDay'].isin(['Saturday', 'Sunday'])]
```

```
# Getting all flights on weekdays
weekday_flights = df[~df['WeekDay'].isin(['Saturday', 'Sunday'])]
```

```
weekend_flights['Price'].mean()
```

```
np.float64(9058.016077170418)
```

```
weekday_flights['Price'].mean()
```

```
np.float64(9015.219666215608)
```

Answer: The average price of flight tickets are on **Weekends**.

Q4. Two of the entries in the 'Additional_Info' column are 'No info' and 'No Info'. Replace all occurrences of 'No Info' with 'No info'. How many flights fall under airline 'IndiGo' and have 'No info' as additional information?

```
df['Additional_Info'].replace('No Info', 'No info', inplace=True)
```

```
df['Additional_Info']
```

```
0       In-flight meal not included
1       In-flight meal not included
2                           No info
3       In-flight meal not included
4                           No info
                  ...
```

```
9445    In-flight meal not included
9446                        No info
9447                        No info
9448                        No info
9449                        No info
Name: Additional_Info, Length: 9450, dtype: object
```

```
df[
    (df['Airline'] == 'IndiGo') &
    (df['Additional_Info'] == 'No info')
].shape[0]
```

```
1650
```

Answer: **1650**

Learning : When using logical operators like & for filtering DataFrame rows, you need to put parentheses around the conditions.

Q5. Convert the values of 'Duration' into seconds. Enter the average duration (in seconds) of a flight. Enter your answer correct to two decimal places.

Below mentioned is the solution to question 5 and also the key-takeaway from the Practice Assignment - 1 section.

Apply the following functions to the columns Dep_Time and Arrival_Time:

Transform the values in the 'dep_time' and 'arrival_time' columns to represent the hour component. For instance, if an entry is 10:05 June 13 or 10:05, the corresponding value should be 10. Then convert the time into four categories as follows:

1. 5 <= hour < 12 = Morning
2. 12 <= hour < 17 = Afternoon
3. 17 <= hour < 20 = Evening
4. 20 <= hour < 5 = Night

Note: Please ensure that you make the changes directly within the dataset and continue to use that modified dataset for subsequent questions.

```
# Extract hour from Dep_Time and Arrival_Time
df['Dep_Time'] = pd.to_datetime(df['Dep_Time'], errors='coerce').dt.hour
df['Arrival_Time'] = pd.to_datetime(df['Arrival_Time'], errors='coerce').dt.
```

```
C:\Users\Adarsh Kumar\AppData\Local\Temp\ipykernel_24512\1471961768.py:2: Use
  df['Dep_Time'] = pd.to_datetime(df['Dep_Time'], errors='coerce').dt.hour
C:\Users\Adarsh Kumar\AppData\Local\Temp\ipykernel_24512\1471961768.py:3: Use
  df['Arrival_Time'] = pd.to_datetime(df['Arrival_Time'], errors='coerce').dt
```

```
def time_category(hour):
    if 5 <= hour < 12:
        return 'Morning'
    elif 12 <= hour < 17:
        return 'Afternoon'
    elif 17 <= hour < 20:
        return 'Evening'
    else:
        return 'Night'

df['Dep_Time'] = df['Dep_Time'].apply(time_category)
df['Arrival_Time'] = df['Arrival_Time'].apply(time_category)
```

```
def duration_to_seconds(x):
    hours = 0
    minutes = 0

    if 'h' in x:
        hours = int(x.split('h')[0])

    if 'm' in x:
        minutes = int(x.split('h')[-1].replace('m', '').strip())

    return hours * 3600 + minutes * 60

df['Duration'] = df['Duration'].apply(duration_to_seconds)

print(df['Duration'].mean())
```

```
38957.93650793651
```

Answer: Average duration of flights in seconds is **38957.94**

Q6. How many flights started in the Morning and arrived the destination at Evening?

```
df[(df['Dep_Time'] == 'Morning') & (df['Arrival_Time'] == 'Evening')].shape[
```

```
922
```

Answer: Total **922** departed in the morning and arrived in the evening

Lab-1 Assignment 2

Loading the dataset

Click here to view the dataset: Dataset

```
df = pd.read_csv('V5.csv')
```

```
df.shape
```

```
(10000, 12)
```

Shape of the dataset is (10000, 12) [Please note that if you do not obtain this shape, it may indicate that the data was not uploaded correctly to the Colab environment.].For simplicity please consider all the unknown values ("?") as null values for this assignment.Consider each question as individual question unless instructed otherwise and use the given csv data to compute all the solutions.

Q1. How many unknown ("?) values are present in the dataset? Remove/Delete unknown ("?) values present in the dataset to make it null value.Remove/Delete - means it will show NAN in place of "?"

Note: If there is no value present in the dataset it is represented as NAN(read pandas documentation for all the other ways to represent null values) in data

```
df.sample(10)
```

	Date	Year	Locality	Estimated Value	Sale Price	Property	Residential	num_
3759	2015-01-04	2015	Norwalk	NaN	1100000.0	Single Family	Detached House	
959	2010-05-28	2010	West Hartford	208700.0	310000.0	Single Family	Detached House	
3659	2014-11-09	2014	Norwalk	NaN	452500.0	Single Family	Detached House	
9746	2022-08-04	2022	Bridgeport	NaN	255000.0	?	Detached House	
1477	2011-06-29	2011	Fairfield	607880.0	840000.0	Single Family	Detached House	
3157	2014-01-27	2014	Waterbury	NaN	31000.0	Single Family	Detached House	

```
(df=='?').sum().sum()
```

```
np.int64(1823)
```

```
df.replace('?', np.nan, inplace=True)
```

```
(df=='?').sum().sum()
```

```
np.int64(0)
```

Answer: Total number of '?' values in the dataset is **1823**

Q2. What is the shape of the dataset ?

```
df.shape
```

```
(10000, 12)
```

Answer: Shape of the dataset is **(10000, 12)**

Q3. What is the value present at the 692th indexed row and 0th indexed column in the data ?

```
df.iloc[692, 0]
```

```
'2009-11-16'
```

Answer: Value at [692, 0] is **'2009-11-16'**

Q4. What is the value present at the 546th indexed row and 7th indexed column in the data ? dataframe[546,7] (simpy saying this in a matrix) rows/ columns starts indexing from zero(0) in python

```
df.iloc[546, 7]
```

```
np.int64(3)
```

Answer: Value at [546, 7] is **3**

Q5. What are the unique values present in the Locality feature of the dataset?

```
len(df['Locality'].unique())
```

```
8
```

Answer: Total unique values present in the 'Locality' feature are **8**

Q6. Which of the following features have missing(NaN) values present in the dataset?

(Note: compute after removing "?")

```
df.isna().sum()[df.isna().sum() > 0].index.tolist()
```

```
['Locality', 'Estimated Value', 'Property', 'carpet_area']
```

Answer: Features having missing (NaN) values are **['Locality', 'Estimated Value', 'Property', 'carpet_area']**

Q7. Which of the following feature has most missing(NaN) values present in the dataset?

(Note: compute after removing "?")

```
df.isna().sum()
```

```
Date                   0
Year                   0
Locality            1253
Estimated Value     1243
Sale Price             0
Property            1823
Residential            0
num_rooms              0
num_bathrooms          0
carpet_area         1229
property_tax_rate      0
Face                   0
dtype: int64
```

Answer: Feature having most missing (NaN) values is **Property**

Q8. Drop all the samples(rows) with missing values strictly greater than 2. How many samples remains after that ?

```
df[df.isna().sum(axis=1) > 2]
```

	Date	Year	Locality	Estimated Value	Sale Price	Property	Residential	num_
194	2009-03-27	2009	NaN	NaN	3900000.0	Single Family	Detached House	
295	2009-05-15	2009	NaN	NaN	6632000.0	NaN	Detached House	
425	2009-06-29	2009	NaN	NaN	1049000.0	Single Family	Detached House	
470	2009-07-17	2009	NaN	NaN	260403.0	NaN	Detached House	
682	2009-11-08	2009	NaN	NaN	1150000.0	Single Family	Detached House	
...	...	...	...	...	...	...	...	
9116	2021-11-26	2021	NaN	NaN	300000.0	NaN	Triplex	
9337	2022-03-04	2022	Waterbury	NaN	58000.0	NaN	Detached House	

```
df[df.isna().sum(axis=1) > 2].shape
```

```
(83, 12)
```

Answer:

- There are total **83** rows with more than 2 missing values

- Total samples remain after this operation is 10000 - 83 = **9917**

Q9. Drop all the samples(rows) with missing values in the original dataframe . How many samples remains after that ?

```
df = pd.read_csv('V5.csv')
```

```
df[df.isna().sum(axis=1) > 0]
```

	Date	Year	Locality	Estimated Value	Sale Price	Property	Residential	num_
6	2009-01-02	2009	Norwalk	571060.0	1000000.0	Single Family	Detached House	
8	2009-01-02	2009	NaN	603540.0	1680000.0	Single Family	Detached House	
11	2009-01-03	2009	Norwalk	NaN	1340000.0	Single Family	Detached House	
14	2009-01-04	2009	Bridgeport	NaN	300000.0	Three Family	Triplex	
18	2009-01-05	2009	Bridgeport	97719.0	265000.0	Two Family	Duplex	
...	...	...	...	...	...	...	...	
9990	2022-09-30	2022	NaN	352030.0	700000.0	?	Detached House	
9991	2022-	2022	Norwalk	344270.0	512500.0	Single	Detached	

```
df[df.isna().sum(axis=1) > 0].shape
```

```
(3300, 12)
```

```
df.isna().sum()[df.isna().sum() > 0]
```

```
Locality          1253
Estimated Value   1243
carpet_area       1229
dtype: int64
```

Answer:

- There are total **3300** rows with more than 2 missing values
- Total samples remain after this operation is 10000 - 3300 = **6700**

Q10. Select all the even indexed rows from the given dataset. What is the value in the 3rd indexed column of the 0th indexed row in the selected dataframe. (row/ column indexed from zero(0) in python/pandas)

```
df.iloc[::2]
```

	Date	Year	Locality	Estimated Value	Sale Price	Property	Residential	num_
0	2009-01-02	2009	Waterbury	111440.0	185000.0	Single Family	Detached House	
2	2009-01-02	2009	Waterbury	55720.0	140000.0	Single Family	Detached House	
4	2009-01-02	2009	Bridgeport	112351.0	210000.0	?	Detached House	
6	2009-01-02	2009	Norwalk	571060.0	1000000.0	Single Family	Detached House	
8	2009-01-02	2009	NaN	603540.0	1680000.0	Single Family	Detached House	
...	...	...	...	...	...	...	...	
9990	2022-09-30	2022	NaN	352030.0	700000.0	?	Detached House	
9992	2022-	2022	Fairfield	NaN	680000.0	Single	Detached	

```
df.iloc[::2].iloc[0, 3]
```

```
np.float64(111440.0)
```

Answer: The value are [0, 3] after selecting even rows is **111440.0**

Selection odd indexed columns

```
df.iloc[1::2]
```

	Date	Year	Locality	Estimated Value	Sale Price	Property	Residential	num_
1	2009-01-02	2009	Bridgeport	124670.0	150000.0	Two Family	Duplex	
3	2009-01-02	2009	Bridgeport	4775276.0	272900.0	Single Family	Detached House	
5	2009-01-02	2009	Greenwich	2868880.0	3300000.0	Single Family	Detached House	
7	2009-01-02	2009	Greenwich	592760.0	920000.0	Single Family	Detached House	
9	2009-01-02	2009	Bridgeport	137901.0	245000.0	Single Family	Detached House	

Selecting even indexed columns

```
df.iloc[::2]
```

	Date	Year	Locality	Estimated Value	Sale Price	Property	Residential	num_
9991	2022-09-30	2022	Norwalk	344270.0	512500.0	Single Family	Detached House	
0	2009-01-02	2009	Waterbury	111440.0	185000.0	Single Family	Detached House	
2	2009-01-02	2009	Waterbury	55720.0	140000.0	Single Family	Detached House	
4	2009-01-02	2009	Bridgeport	112351.0	210000.0	?	Detached House	
6	2009-01-02	2009	Norwalk	571060.0	1000000.0	Single Family	Detached House	